Digital System Design Report 3

Group 29

Authors: Qichao (Arlo) Wang (qw823)

Table 1: Baseline adopted from Report 2 (V6).

Metric	C2	C3	C4
Latency (ms/run)	137.0	4380.0	165.0
ALMs		2,103	
DSP blocks		6	

1 Introduction and Methodology

This report covers Task 7 (7a/7b/7c) and Task 8 of the coursework. For an input vector $X = \{x_0, x_1, \ldots, x_{N-1}\}$, the target computation is

$$F(X) = \sum_{k=0}^{N-1} f(x_k), \quad f(x) = 0.5x + x^3 \cos\left(\frac{x - 128}{128}\right)$$

under single-precision floating-point representation.

To keep the report structure consistent with Report 2, this document also introduces an explicit baseline and then reports Task-7/Task-8 results as incremental evolution from that baseline.

1.1 Baseline from Report 2

The baseline used in this report is the Report-2 V6 configuration (FP custom add/sub/mul with `cosf` in software flow). This baseline is selected to provide a consistent pre-CORDIC reference point for Task-7 and Task-8 comparisons.

Methodology summary: deterministic vectors C2 ($N = 2041$), C3 ($N = 65{,}281$), and random C4 ($N = 2323$, fixed seed) are evaluated with 10 runs per case. C1 is omitted because its timing is below one tick at 1 kHz resolution in later implementations. Reported metrics are latency, relative error versus double-precision software reference, and Quartus resource/timing closure at 50 MHz.

2 Task 7a: CORDIC Analysis

2.1 Reference Model and Sweep

Task 7a uses a MATLAB Monte Carlo model with $\theta \sim U[-1, 1]$ (`single`), compares CORDIC output to single-precision $\cos(\theta)$, and reports MSE with 95% confidence bounds. The sweep uses all integer $N_{iter} \in [12, 24]$ and FRAC$\in [20, 34]$ with $W = \text{FRAC} + 2$.

2.2 Selection Result

The acceptance criterion is:

$$\text{CI}_{95,\text{upper}}(\text{MSE}) < 2.4 \times 10^{-11}.$$

After including odd values, the first passing point is

$$(N_{iter}, W, \text{FRAC}) = (17, 23, 21).$$

The implementation uses $(18, 24, 22)$ with `ITER_PER_CYCLE`=3 as a conservative point: it keeps the same CORDIC cycle count as 17 iterations under IPC=3 while providing larger error margin.

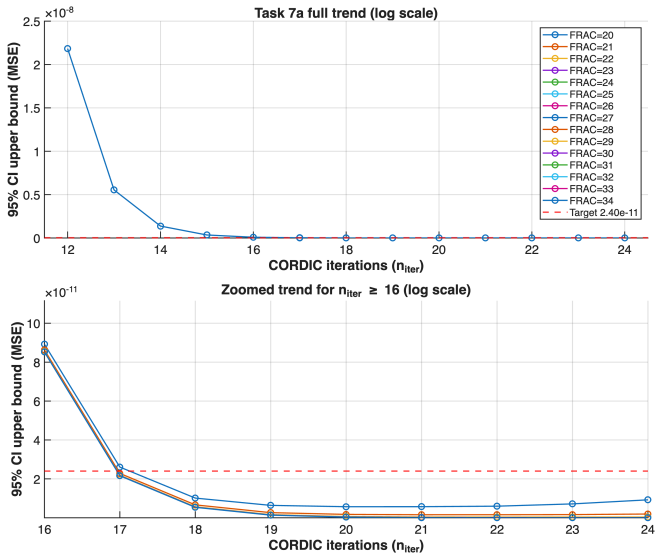

Figure 1: 95% CI upper bound of MSE versus iterations for each fixed-point precision (top: full sweep; bottom: zoom for $N_{iter} \geq 16$, both on log scale).

3 Task 7b: CORDIC Architecture and Implementation

3.1 CORDIC Microarchitecture

The cosine block is rotation-mode fixed-point CORDIC with start/busy/done handshake. Final hardware parameters are $N_{iter} = 18$, FRAC= 22, width $W = 24$, and `ITER_PER_CYCLE`=3, so the compute stage takes $\lceil 18/3 \rceil = 6$ cycles. Increasing IPC improves cosine-block latency, but Step-2 end-to-end gain is limited by software orchestration overhead.

3.2 Verification and Timing

Standalone CORDIC simulation at $\theta \in \{-1, -0.5, 0, 0.5, 1\}$ passed with maximum absolute error 3.37×10^{-6}. The final build uses 3-cycle FP add/sub/mul latency (instead of the earlier 2-cycle attempt) to avoid integration mismatch. Post-fit timing remains closed at 50 MHz (setup slack 3.106 ns, hold slack 0.086 ns, Fmax 59.19 MHz).

4 Task 7c: Single Custom Instruction Integration

4.1 Step-2 vs Step-3 Integration

Step-2 uses separate FP_MUL, FP_ADD/SUB, and COS custom instructions with software scheduling. Step-3 integrates the full inner function into one custom instruction:

$$f(x) = 0.5x + x^3 \cos\left(\frac{x - 128}{128}\right).$$

The Step-3 block reuses one FP multiplier, one FP adder, and one CORDIC core under an FSM, reducing CPU-visible calls from multiple operators per sample to one call per sample.

Table 2: Task 7c summary across C2/C3/C4 (average per run, 10-run measurements).

Metric	C2	C3	C4
Step-2 latency (ms/run)	63.7	2036.7	72.4
Step-3 latency (ms/run)	7.0	220.0	8.0
Speedup (Step-2 / Step-3)	9.10×	9.26×	9.05×
Step-2 speedup vs R2 baseline	2.15×	2.15×	2.28×
Step-3 speedup vs R2 baseline	19.57×	19.91×	20.62×
Relative error vs double	5.001e-07	2.357e-05	2.135e-07

Table 3: Resource/timing comparison of Step-2 and final Step-3 integration.

Metric	Step-2	Step-3
ALMs	3767	3795
Registers	4207	4203
Memory bits	3,159,680	3,159,680
RAM blocks	397	397
DSP blocks	1	1
Worst setup slack (ns)	3.106	2.907
Worst hold slack (ns)	0.086	0.085
Fmax `sopc_clk` (MHz)	59.19	58.5

4.2 Performance, Resource, and Error Results

Step-3 adds only 28 ALMs over Step-2 while preserving timing closure at 50 MHz.

4.3 Latency Interpretation

Isolated-cycle estimates are lower than Nios-integrated measurements. In Step-2, one sample evaluation requires five FP multiplications, one subtraction, one addition, and one cosine call:

$$T_{\text{Step-2,iso}} = 5 \cdot 3 + 1 \cdot 3 + 1 \cdot 3 + 1 \cdot 9 = 30 \text{ cycles/sample.}$$

For C3, this predicts about 39.2 ms/run at 50 MHz versus measured 2036.7 ms/run. Step-3 isolated latency is 21 cycles (about 27.4 ms/run idealized for C3) versus measured 220.0 ms/run. The remaining gap is mainly software/interface overhead, but Step-3 significantly reduces it by collapsing call granularity.

4.4 Ideal Lower Bound (Dummy Combinational CI)

The dummy one-cycle CI was not implemented in this submission; therefore this subsection reports the theoretical lower bound only. If one $f(x)$ evaluation took exactly one clock cycle, C3 (65,281 samples) would require 65,281 cycles per run, i.e. 1.306 ms at 50 MHz. For the measured 10-run setup, this gives a lower bound of 13.06 ms total, versus the measured Step-3 total of 2200 ms. This gap confirms that the current bottleneck is system-level integration overhead rather than arithmetic datapath depth alone.

5 Task 8: Throughput-Oriented Final Design

Task 8 targets throughput by moving accumulation into the CI datapath, so each loop iteration performs one accumulator update call. The deployed run uses the current two-operand accumulate interface and reuses Task-7 arithmetic blocks.

5.1 Task 8 Results

The C4 run follows the same trend in timing (44 total ticks over 10 runs, i.e. 4.4 ms/run) and shows the same correctness issue as C2/C3. In this dataset Step-4 outputs are zero, so latency gains are integration-stage observations, not valid functional acceleration results.

Table 4: Task-8 Step-4 measured software results (10 runs, 1 kHz tick source).

Metric	C2	C3	C4
Latency of $F(x)$ (ms/run)	3.8	123.5	4.4
Speedup vs Step-3 latency	1.84×	1.78×	1.82×
Speedup vs R2 baseline	36.05×	35.47×	37.50×
Relative error vs double	1.000e+00	1.000e+00	1.000e+00
Code size (application image)	N/A	N/A	N/A

Table 5: Task-8 post-fit resource/timing snapshot (current build).

Metric	Value
ALMs	3,793
Registers	4,214
Block memory bits	3,159,680
RAM blocks	397
DSP blocks	1
Worst setup slack (ns)	3.205
Worst hold slack (ns)	0.114
Fmax `sopc_clk` (MHz)	59.54

Compared with Task 7 Step-3, this build remains in essentially the same resource class and keeps timing closure at 50 MHz with positive slack.

6 Cross-Task Comparison and Discussion

6.1 Latency Comparison from Baseline to Task 8

6.2 Key Trade-offs

The baseline comparison clarifies the core trade-off. Relative to the Report-2 V6 baseline, Task-7 Step-2 already improves latency because the CORDIC-based cosine path is cheaper than the software `cosf` bottleneck in V6. Task-7 Step-3 then collapses the inner arithmetic sequence into one custom instruction call per sample and gains a much larger system-level improvement, reaching about 19.91× speedup on C3 relative to baseline while preserving correctness within the measured error bound. This confirms that both algorithm choice (CORDIC replacing `cosf`) and call granularity are decisive in the Nios-integrated setting.

Task-8 Step-4 further reduces reported run time, but current hardware/software integration still shows functional mismatch in the measured dataset (relative error 1.0). Therefore its timing numbers are best interpreted as integration-stage throughput indicators, not final valid acceleration results. In resource terms, the Task-7/Task-8 builds remain in a similar area class and continue to close timing at 50 MHz, so the remaining bottleneck is primarily interface/control behavior rather than FPGA timing closure.

7 Conclusions

Using the Report-2 V6 result as baseline, the final validated Task-7 architecture shows that the most effective gain comes from combining CORDIC-based cosine with reduced CPU-visible call count. The CORDIC precision study in Task 7a identifies a minimum passing point and a

Table 6: Cross-stage latency comparison against the Report-2 baseline.

Stage	C2 (ms/run)	C3 (ms/run)	C4 (ms/run)	C3 speedup vs baseline
Report-2 V6 baseline	137.0	4380.0	165.0	1.00×
Task-7 Step-2	63.7	2036.7	72.4	2.15×
Task-7 Step-3	7.0	220.0	8.0	19.91×
Task-8 Step-4 (timing only)	3.8	123.5	4.4	35.47×

conservative implementation point, and the implemented Task-7 Step-3 single-CI flow reaches about 19.91× C3 speedup versus baseline while maintaining acceptable numerical accuracy and 50 MHz timing closure.

Task-8 measurements indicate additional latency reduction potential beyond Task-7 Step-3, but the current onboard dataset is not yet functionally correct, so those gains cannot be claimed as final performance. The main remaining engineering target is to preserve Task-8 throughput behavior while restoring functional correctness under the integrated CI interface. Once correctness is closed, the same baseline framework used here can directly produce a final validated speedup figure for submission.